

Motion planning with doors makes games hard

Manuel Frohn

RWTH Aachen University, Aachen, Germany
`manuel.frohn@rwth-aachen.de`

Abstract.

Keywords: PSPACE · Doors · Motion Planning · Games

1 Introduction

Many games, such as the Jump'n Runs of the Super Mario series, require the player to move from one point to another to complete a level. This kind of problem is referred to as 1-Player-Motion-Planning. In simple environments, where nothing but paths exist, this problem is easy, as it is equivalent to the well known s-t-Path Problem. But for most games it is not. One of the reasons for this, is the occurrence of doors devices in the games environment. A door device is a gadget, that consists out of three traversable tunnels and an inner state. One of the tunnels opens the door, one closes it and one acts as the "traverse" tunnel. This tunnel may only be traveled if and only if the door is opened. If such a device is part of the environment of a game we know the game to be PSPACE-hard i.e. An algorithm calculating a path as required by a game will do so by using a minimum of Polynomial-Space in relation to the input length.

2 Fundamentals

2.1 The Problem

The problem of concern here is 1-player-motion-planning (1). It is central to understand that 1-player-motion planning on games does not only look at real levels of the game but rather on any level possible in the games implied environment. This means any level is considered, that may be constructed out of the objects making up the game, like platforms, enemies, or general game mechanics such as gravity changes. Comparable, to how the s-t-path problem on general graphs does not only look at certain graphs but rather at all possible general graphs.

Definition 1 (1-Player-Motion-Planning).

1-Player-Motion-Planning is the problem of finding a path from a given point A to a given point B in a certain environments (such as a graph or a grid) that can be traversed by the games player.

2.2 The Concern

The main concern of this article is PSPACE-hardness. Just as with time complexity, which concerns itself with the runtime of an algorithm, space complexity concerns itself with the space used by an algorithm. And just as with time complexity, space complexity is divided into different complexity classes. The main one of concern here is the class of PSPACE-hard (2) problems. The problems that require **more** than polynomial space to be solved.

Definition 2 (PSPACE-hardness).

*A problem is PSPACE-hard if and only if, the amount of space a optimal algorithm requires to solve the problem is at **least** polynomial in relation to the input length.*

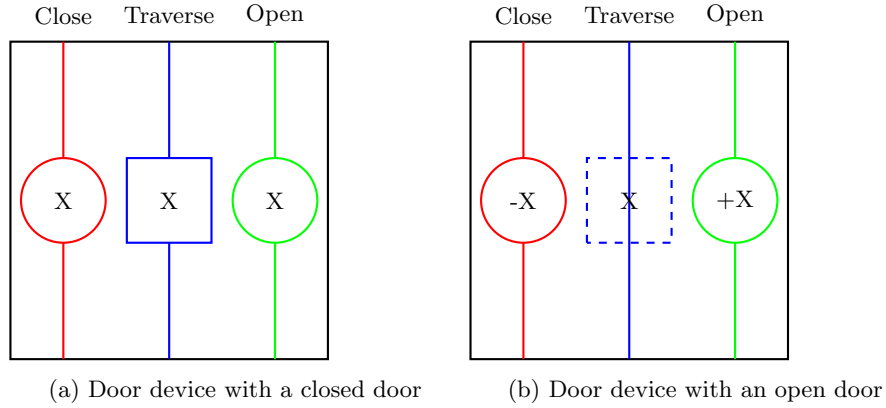
And just as NP has SAT as its canonical problem PSPACE also has a canonical problem, the quantified version of SAT (3) to be precise. There, not only the usage of the logical-or and logical-and is allowed, but also the usage of the quantifiers exists ($\exists$) and forall ($\forall$).

Definition 3 (Quantified-SAT).

Quantified-SAT is the canonical problem of PSPACE. It is an extension of the original SAT problem, expanding the problem by additionally allowing quantified-boolean-algebra expressions. For example $\exists x \forall y : x \wedge y \wedge z$

3 The door device

A door device is a construct made out of three main components: A traverse tunnel, that may only be traversed if a door within it is open, an open tunnel, with a pressure plate which opens the door on being traversed and a close tunnel with a pressure plate which closes the door on being traversed.



4 PSPACE-hardness of doors